Merging and joining data

Hello again! You've learned a lot about pandas, that it's a powerful library that makes working with tabular data easier and more efficient, how to select and index data in a dataframe, how to filter data using Boolean masks, and how to group and aggregate data to derive insights from the story it's telling. In this video, you'll learn how to add new data to existing dataframes. This is a common task for data professionals, but it's not as simple as just adding two dataframes together. There are important considerations to be aware of. By the end of this lesson, you'll have a good understanding of what these considerations are so you can make informed decisions about how best to add data to your project. We're going to learn about two pandas functions: concat and merge. There's considerable overlap between the capabilities of these functions, but it's most important that you learn the basics of each because you will encounter them regularly as a data professional. We'll start with the concat function. Recall that to concatenate means to link or join together. The pandas concat function combines data either by adding it horizontally as new columns for existing rows, or vertically as new rows for existing columns. It's also capable of handling many data-specific complexities that arise, which allows for a high degree of user control. In this video, I'll demonstrate how to use the concat function to add new rows to existing columns, but remember, there's

plenty of support documentation if you'd like additional information. Pandas has a specific way to indicate which way we want the data to be concatenated.

We do this by referring to axes. In fact, many pandas and NumPy functions have an "axis" keyword so you can specify whether you want to apply the function across rows or down columns. The two axes of a dataframe are zero, which runs vertically over rows; and one, which runs horizontally across columns. We'll use our basic planets dataset to demonstrate how concat works. This data has four planets, their radii, and their number of moons, but it's missing the data for Jupiter, Saturn, Uranus, and Neptune. Now suppose we want to add this data, which exists as a separate dataframe. Let's examine this second dataset with information about Jupiter, Saturn, Uranus, and Neptune before joining them.

Notice that this data is in the same format as the data in the df1 dataframe. It has the same columns for planet, radius, and moons. To combine the two dataframes, we'll want to add df2 as new rows below df1. To concatenate the first dataset with information about Mercury, Venus, Earth, and Mars with the second, which has information about Jupiter, Saturn, Uranus, and Neptune, we call PD concat and insert a list of the dataframes we want to concatenate. Then we need to include an axis keyword argument. This instructs the function to combine the data either side-by-side or one on top of the other. We want our resulting dataframe to have eight rows and three columns, which means we want to combine the data vertically.

In other words, we want to add new data by extending axis zero, the vertical axis. Perfect! The data was added as new rows. Notice that each row retains its index number from its original dataframe. If you want the numbering to restart, just reset the

index. We include the "drop equals true" argument because otherwise a new index column will be added to the dataframe, which we don't want in this case. Now the enumeration of the row indices goes from zero to seven.

The concat function is great for when you have dataframes containing identically formatted data that simply needs to be combined vertically. If you want to add data horizontally, consider the merge function. The merge function is a pandas function that joins two dataframes together. It only combines data by extending along axis one horizontally. Let's return to the planets. Now we have the radius and number of moons for all eight planets, but suppose we want to add the data for the planet type, whether it has rings, its average temperature, whether it has a magnetic field, and whether it has life on it. Perhaps this data exists as a separate dataframe, but it's missing Mercury and Venus and it has some recently discovered planets from other star systems, Janssen and Tadmor.

That's okay. We can still work with this. First, let's conceptualize how data joins work. For two datasets to connect, they need to share a common point of reference. In other words, both datasets must have some aspect of them that is the same in each one. These are known as keys. Keys are the shared points of reference between different dataframes, what to match on.

In our case, the keys are the planets. Each dataframe contains planets for us to match on. Now let's consider the different ways that we can join this data. We can join it so only the keys that are in both dataframes get included in the merge. This is known as an inner join. Alternatively, we can join the data so all of the keys from both dataframes get included in the merge. This is known as an outer join.

We can also join the data so all of the keys in the left dataframe are included, even if they aren't in the right dataframe. This is called a left join. Finally, we can join the data so all the keys in the right dataframe are included, even if they aren't in the left dataframe. This is called a right join. Let's examine how each type of join affects our planet data. First we'll call the function and enter df3 and df4 as the left and right positional arguments, respectively. Then we include the keyword argument "on," which lets us specify what our keys to match on should be.

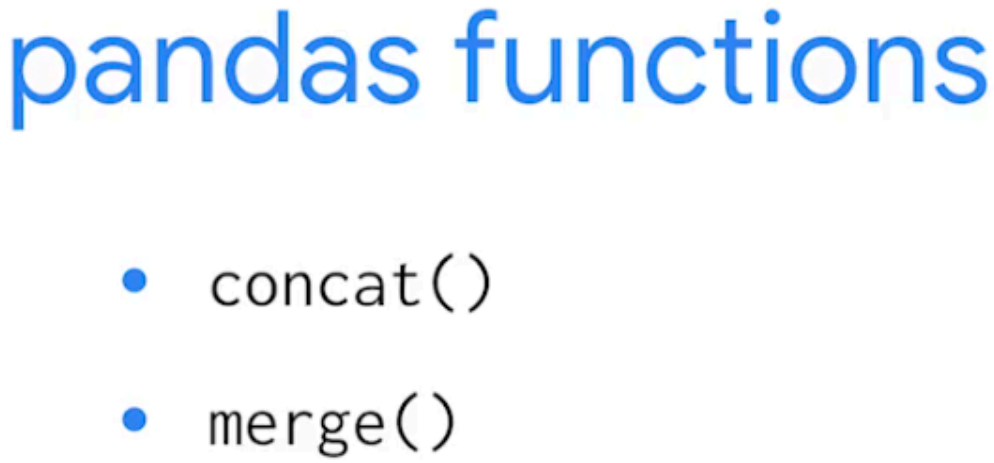
In this case, we want to use the "planet" column. Now we have the "how" keyword argument. This is where we enter the kind of join we want. Let's try "inner" first. This merged the data and only kept the planets that appeared in both dataframes. This means we're missing data for Mercury and Venus from the left dataframe as well as for Janssen and Tadmor from the right dataframe. Now let's try an outer join.

Our function call will remain the same except for the "how" keyword argument, which we'll set to "outer." As expected, this results in a dataframe that contains all the keys from both initial dataframes. Notice that, because Janssen and Tadmor aren't in the left dataframe, they don't have information for radius and moons, so these columns get filled in with NaNs. Similarly, because Mercury and Venus aren't in the right dataframe, they too are missing some information in the final table, which is represented by NaNs.

Next we'll do a left join. Again, the function gets the same syntax except for the "how" argument, which is set to "left." This results in a dataframe that retains all the keys from the left dataframe and only the keys from the right dataframe that exist in the left dataframe too.

So Janssen and Tadmor are excluded. Finally, we'll perform a right join. As expected, the result is a dataframe that has all the keys from the right dataframe, but none of the keys from the left that weren't also in the right. So Mercury and Venus are excluded. Nice job! Now that you know the fundamentals, you can use these pandas tools to do the most common kinds of data joins, which will be useful for a wide variety of data projects. And as you advance in your career, you'll discover even more about joining data, and how it can get very complex.

These tools will be a big help as you do. You've come a long way and are now ready to start using pandas to explore your data like a true data professional. See you soon!



concat()

A pandas function that combines data either by adding it horizontally as new columns for existing rows, or vertically as new rows for existing columns

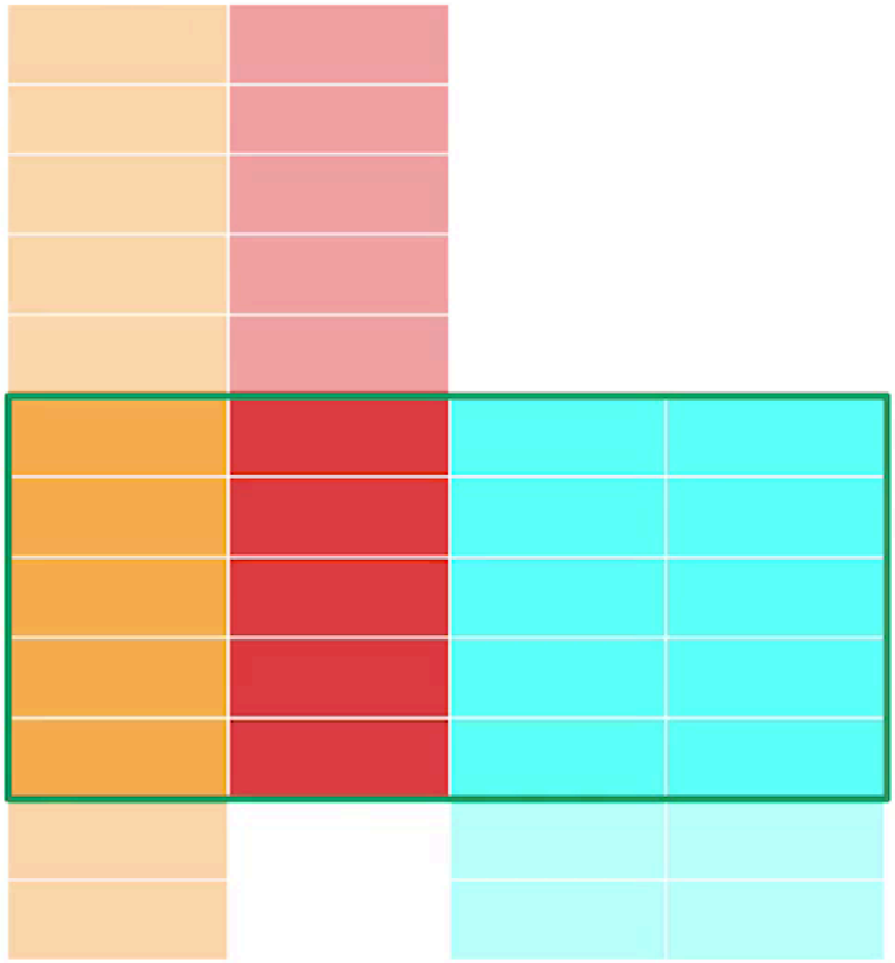
`merge()`

A pandas function that joins two dataframes together; it only combines data by extending along axis one horizontally

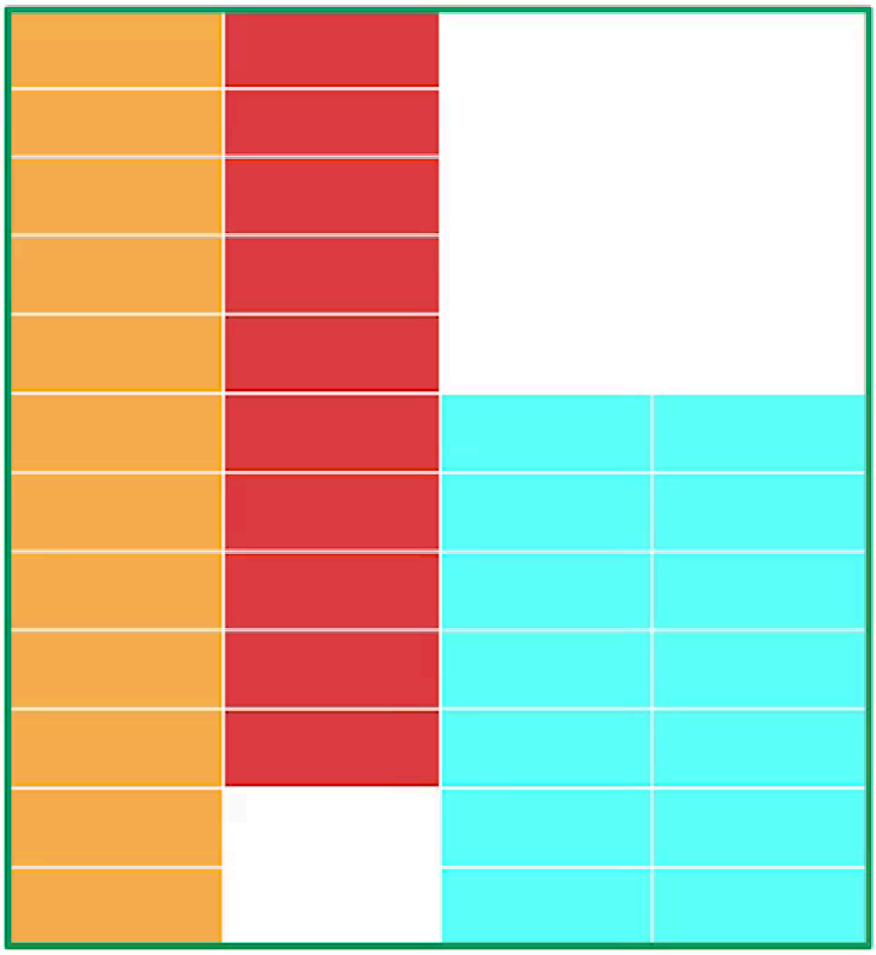
Keys

The shared points of reference between different dataframes—what to match on

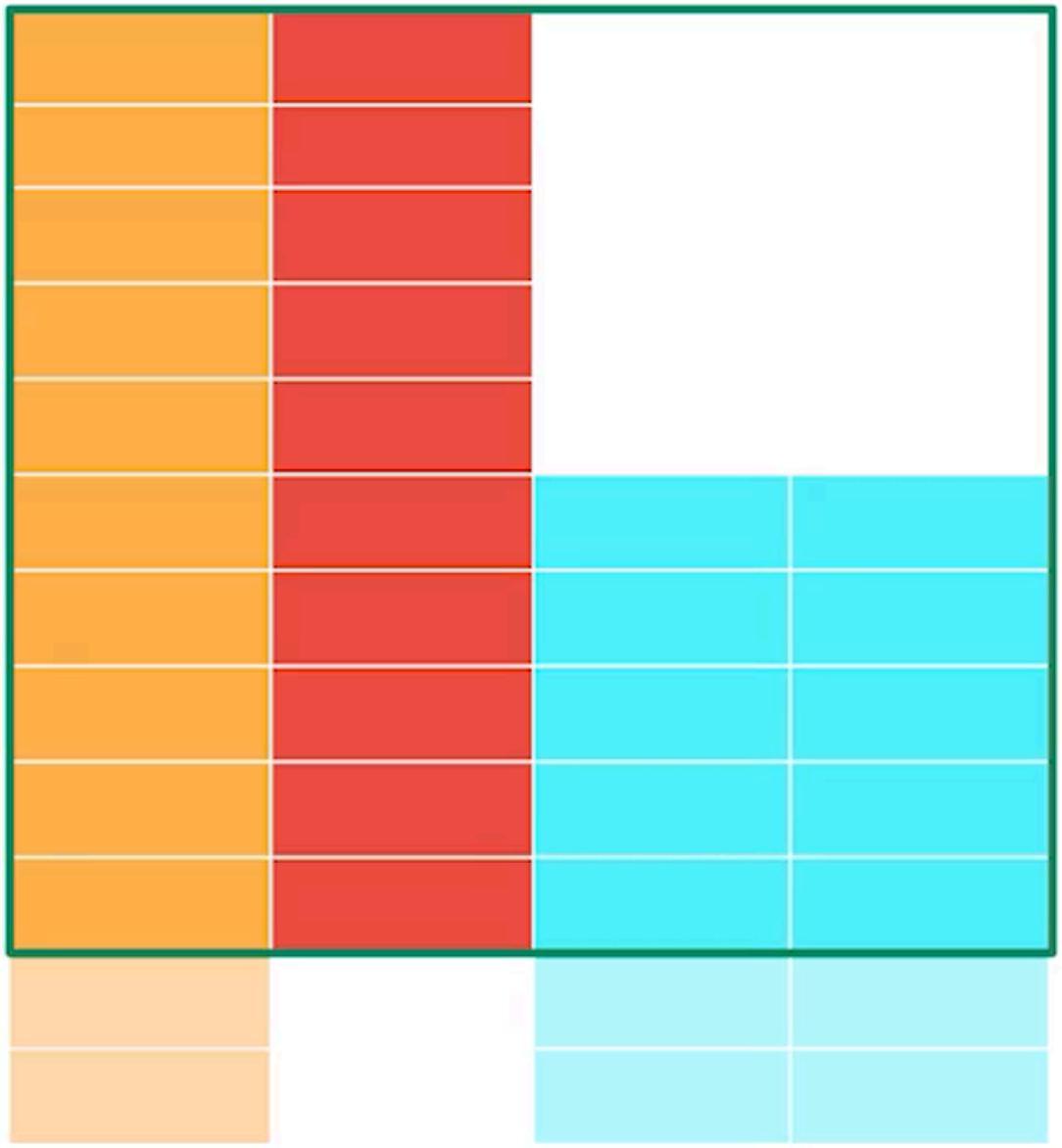
how='inner'



how='outer'



how='left'



how='right'